

رسالة محمد

یادگیری ماشین

مدرس: محمدرضا محمدی

بهار ۱۴۰۲

یادگیری گروهی و جنگل تصادفی

Ensemble Learning and Random Forests

یادگیری گروهی

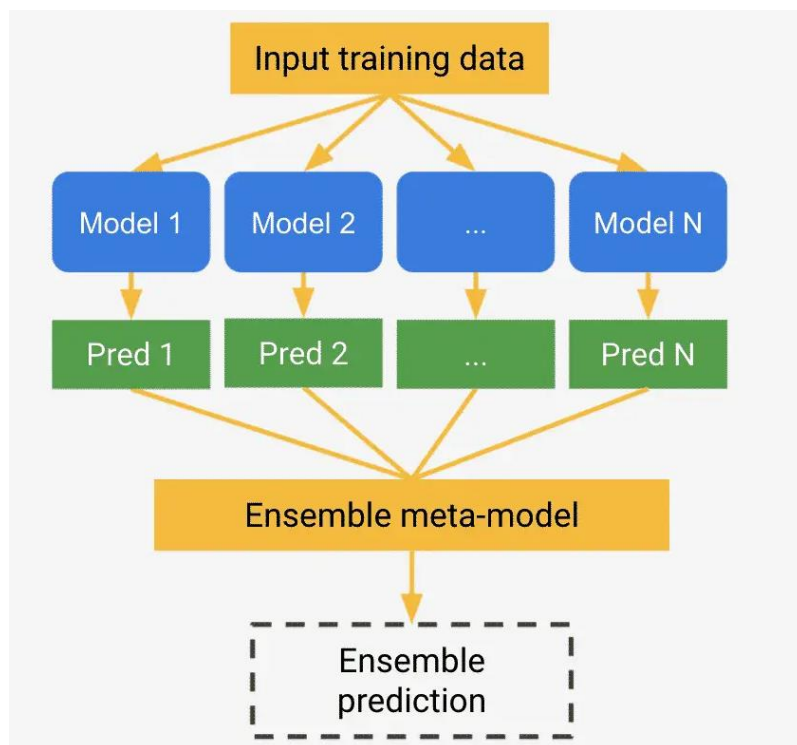
- فرض کنید یک سوال پیچیده از هزاران انسان به صورت تصادفی پرسیده شود و پاسخ آنها تجميع شود
- در بسیاری از مواقع، این پاسخ از پاسخ یک انسان خبره هم بهتر است

- حکمت جمعیت (wisdom of the crowd)

- در یادگیری ماشین هم می توان پیش بینی های گروه هایی از پیش بینی کننده ها را تجميع کرد

- اغلب پیش بینی های بهتری نسبت به بهترین پیش بینی کننده فردی خواهد داشت

- گروهی از پیش بینی کننده ها یک ensemble نامیده می شوند



Bagging and Pasting

- یک راه دیگر برای دستیابی به مدل‌های متنوع این است که یک الگوریتم یادگیری ثابت (مانند درخت تصمیم) بر روی زیرمجموعه‌های متفاوتی از مجموعه آموزشی آموزش ببیند

- اگر نمونه‌برداری با جایگزینی انجام شود، به آن bagging (bootstrap aggregating) گفته می‌شود

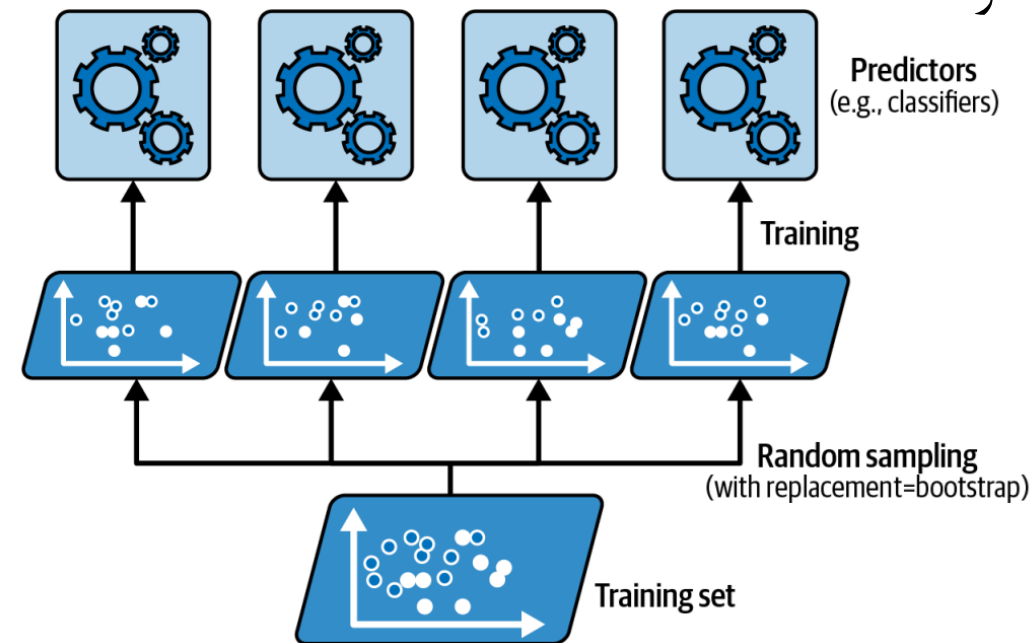
- اگر نمونه‌برداری بدون جایگزینی انجام شود، به آن pasting گفته می‌شود

- در هر دو حالت، برای هر پیش‌بینی‌کننده، نمونه‌برداری از مجموعه اولیه انجام می‌شود

- پیش‌بینی مدل‌ها با یک روش آماری ترکیب می‌شود (رای‌گیری برای دسته‌بندی و میانگین برای رگرسیون پرکاربرد است)

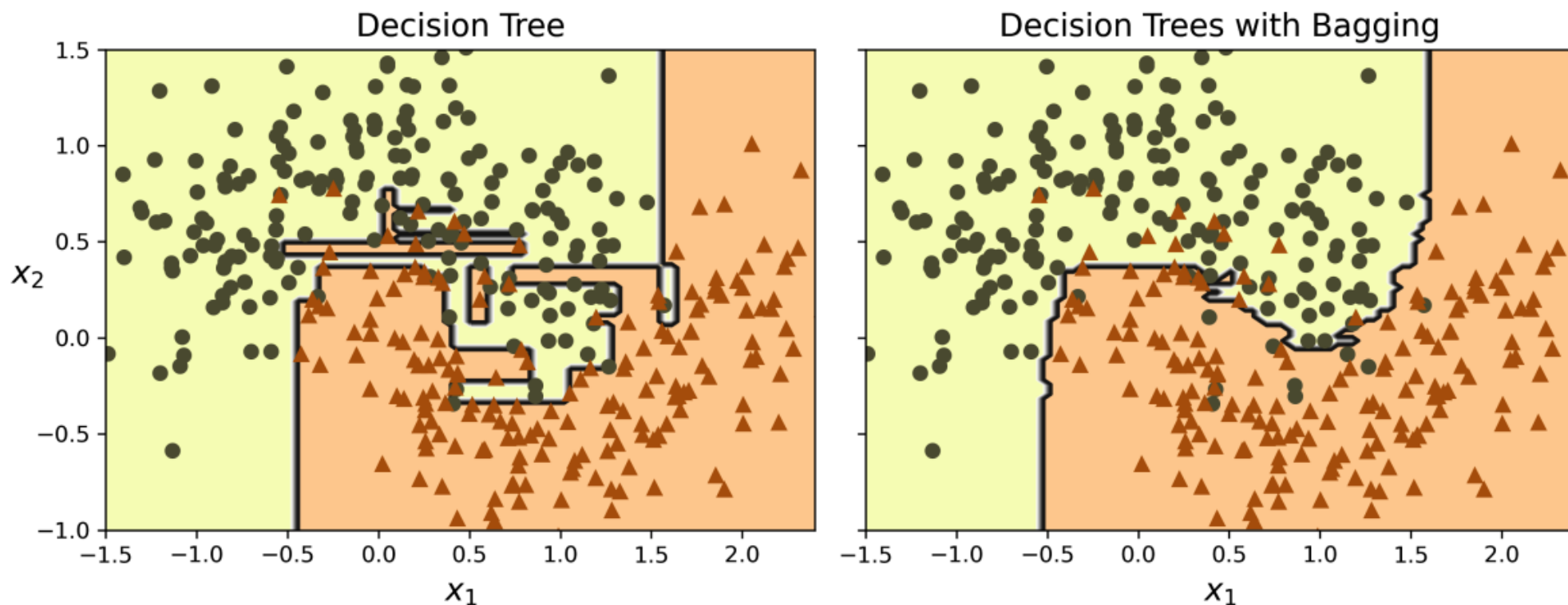
- بایاس حاصل مشابه با آن است که یک مدل روی تمام مجموعه آموزشی آموزش داده شود اما واریانس آن کمتر است

- پیش‌بینی‌کننده‌ها می‌توانند به صورت موازی اجرا شوند



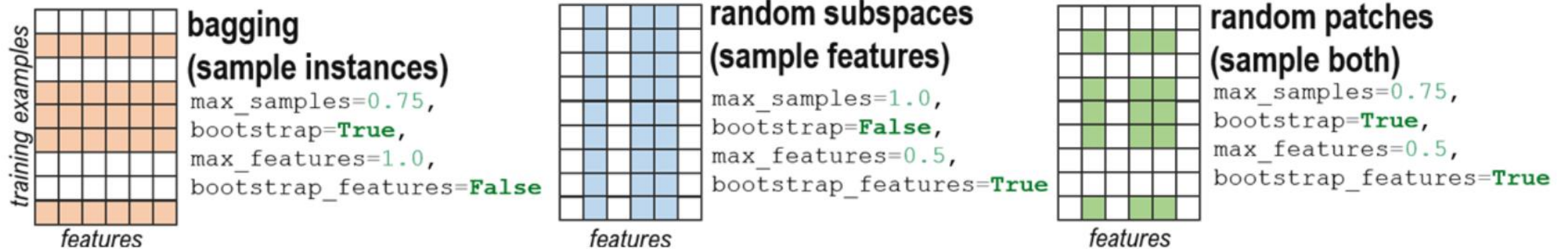
```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                             max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```



زیرفضاهای تصادفی و تکه‌های تصادفی

- می‌توان برای آموزش مدل از ویژگی‌ها هم نمونه‌برداری کرد و هر پیش‌بینی‌کننده روی یک زیرمجموعه از ویژگی‌ها آموزش ببیند
- اگر در آموزش هر مدل از تمام نمونه‌ها و زیرمجموعه‌ای تصادفی از ویژگی‌ها استفاده شود، زیرفضاهای تصادفی (random subspaces) نامیده می‌شود
- اگر در آموزش هر مدل از زیرمجموعه‌ای تصادفی از نمونه‌ها و زیرمجموعه‌ای تصادفی از ویژگی‌ها استفاده شود، تکه‌های تصادفی (random patches) نامیده می‌شود



جنگل تصادفی

- در جنگل تصادفی، گروهی از درخت‌های تصمیم آموزش داده می‌شود و پیش‌بینی آنها تجمیع می‌شود

- معمولاً از ایده bagging استفاده می‌کند

```
from sklearn.ensemble import RandomForestClassifier
```

```
rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,  
                                n_jobs=-1, random_state=42)
```

```
rnd_clf.fit(X_train, y_train)
```

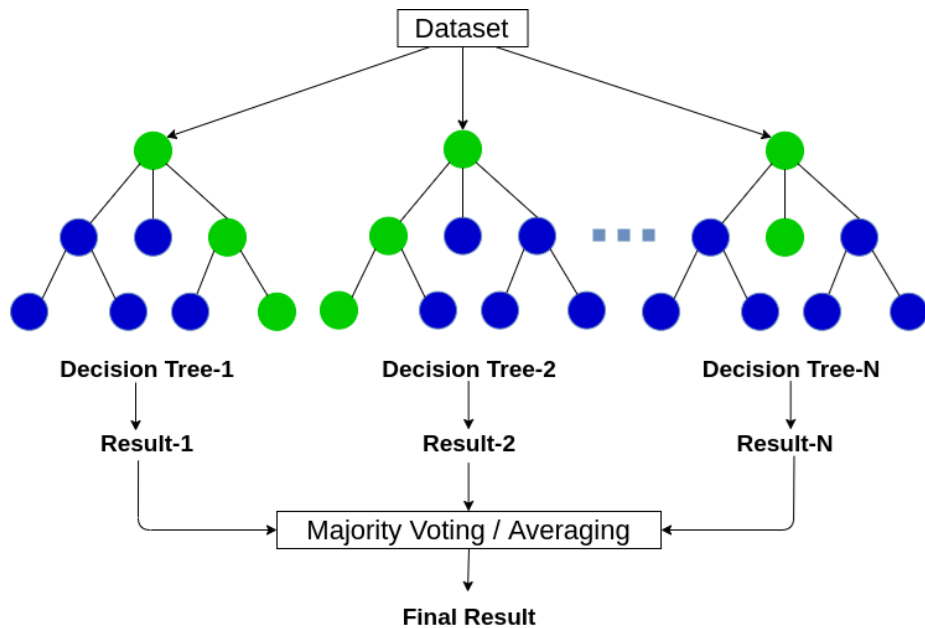
```
y_pred_rf = rnd_clf.predict(X_test)
```

- در هر گره، برای تعیین ویژگی متمایزگر، تنها از زیرمجموعه‌ای از ویژگی‌ها استفاده می‌کند که تصادفی بودن و تنوع مدل‌ها را افزایش می‌دهد

- به صورت پیش‌فرض، ویژگی را از میان $\sqrt{n}$ ویژگی تصادفی از مجموع n ویژگی انتخاب می‌کند
- این کار باعث تنوع بیشتر، بایاس بیشتر و واریانس کمتر می‌شود

اهمیت ویژگی‌ها (Feature Importance)

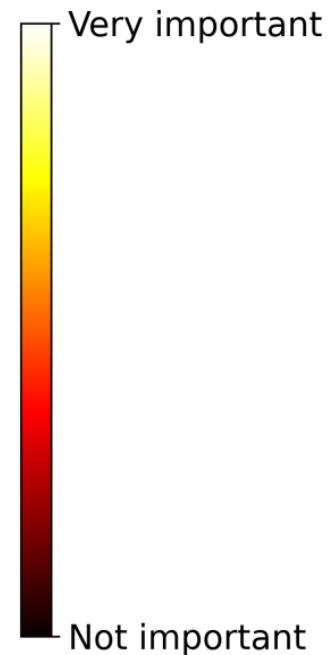
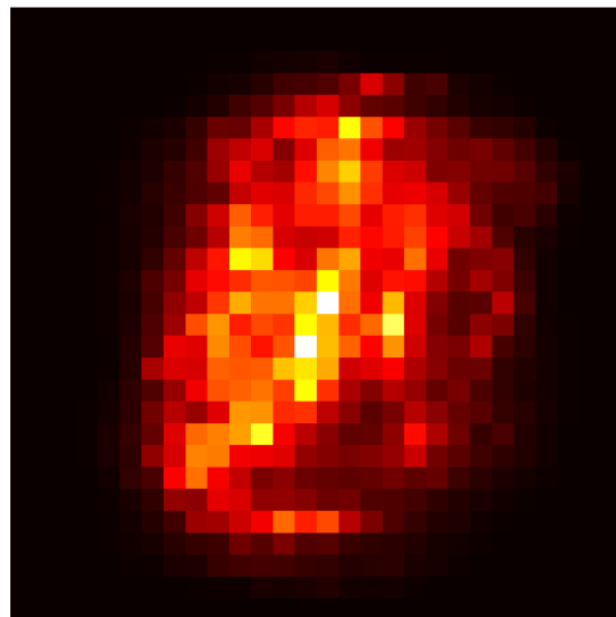
- میزان استفاده شدن ویژگی‌ها در درخت‌های تصمیم و اثرگذاری آنها در کاهش ناخالصی می‌تواند به عنوان معیاری برای سنجش اهمیت ویژگی‌ها برای حل مسئله در نظر گرفته شود
- برای هر DT، اهمیت هر node برابر با کاهش میزان ناخالصی در نظر گرفته می‌شود و اهمیت ویژگی برابر با مجموع اهمیت nodeهایی است که از آن ویژگی استفاده می‌کنند
 - تعداد نمونه‌های هر node نیز در محاسبه مجموع در نظر گرفته می‌شود
- میانگین اهمیت هر ویژگی در تمام درخت‌ها به عنوان اهمیت آن در جنگل تصادفی لحاظ می‌شود
- در نهایت، این مقادیر نرمالیزه می‌شوند که مجموع برابر با ۱ شود



اهمیت ویژگی‌ها

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
```

```
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

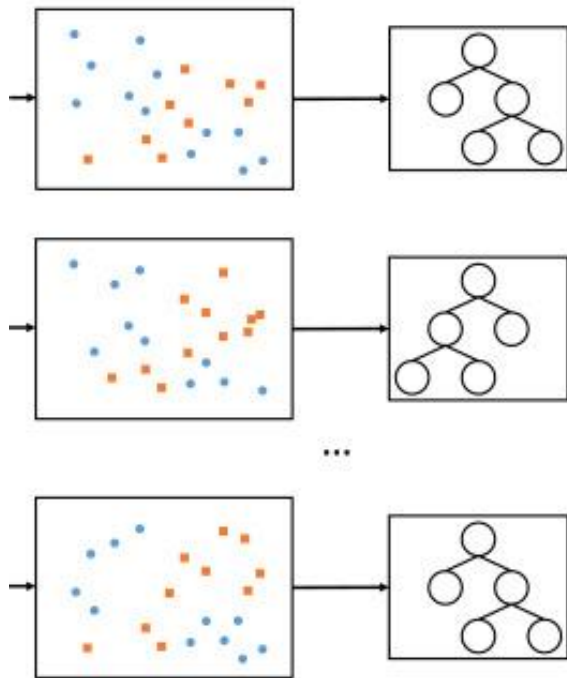


- برای مجموعه داده MNIST:

Boosting

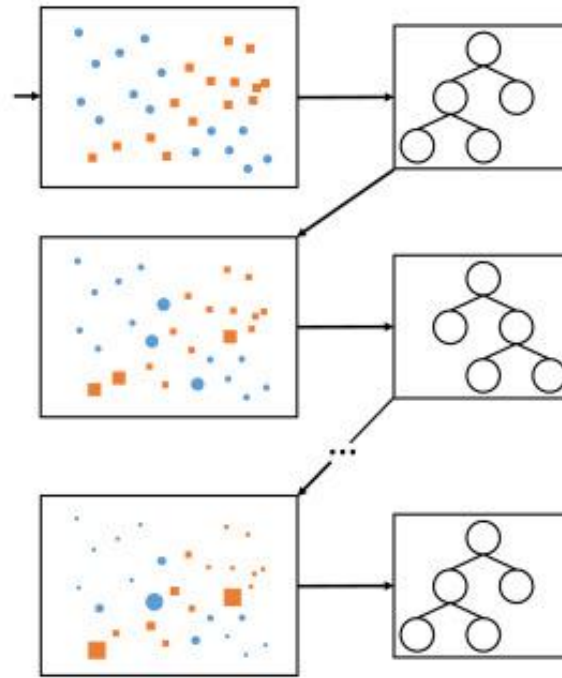
- ایده کلی اغلب روش‌های Boosting این است که پیش‌بینی‌کننده‌ها به صورت ترتیبی آموزش داده شوند و هر کدام تلاش می‌کنند قبلی‌ها را اصلاح کنند

Bagging



Independent weak learner training

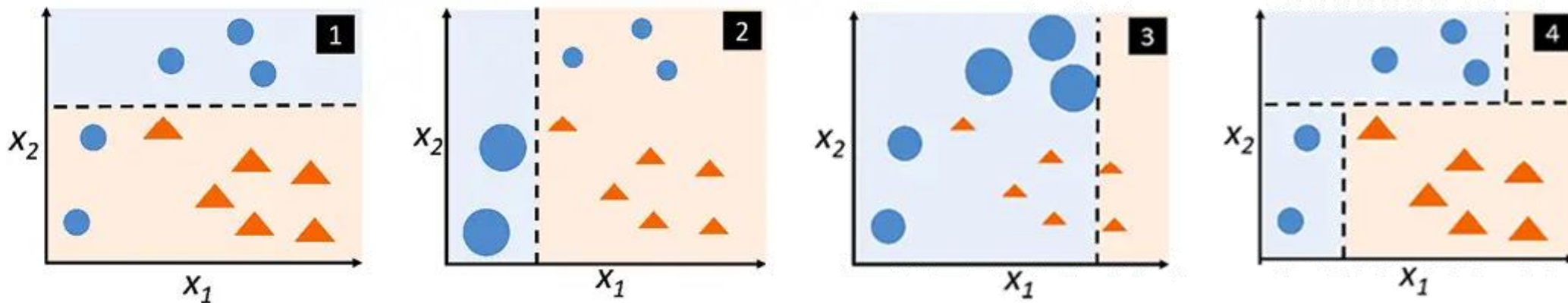
Boosting



Iterative weak learner training

AdaBoost

- در AdaBoost (Adaptive Boosting)، هر پیش‌بینی‌کننده جدید توجه بیشتری به نمونه‌هایی اختصاص می‌دهد که پیش‌بینی‌کننده‌های قبلی در مورد آنها اشتباه می‌کنند
 - پیش‌بینی‌کننده‌ها بیشتر و بیشتر بر نمونه‌های سخت تمرکز می‌کنند
 - شباهت‌هایی به الگوریتم گرادیان کاهشی دارد اما بجای آنکه پارامترهای یک پیش‌بینی‌کننده را برای مینیمم کردن تابع هزینه تنظیم کند، پیش‌بینی‌کننده جدید اضافه می‌کند تا پیش‌بینی را بهتر کند
 - پس از آموزش، پیش‌بینی نهایی را مشابه با Bagging انجام می‌دهد اما هر پیش‌بینی‌کننده وزن متفاوتی دارد



AdaBoost

$$w^{(i)} = \frac{1}{m}$$

$$r_j = \sum_{i=1}^m w^{(i)} (\hat{y}_j^{(i)} \neq y^{(i)})$$

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

$$w^{(i)} \leftarrow w^{(i)} / \sum w^{(i)}$$

- در ابتدا وزن تمام نمونه‌ها یکسان قرار داده می‌شود
- پیش‌بینی‌کننده اول آموزش داده می‌شود
- نرخ خطای وزن‌دار آن به صورت زیر محاسبه می‌شود:
 - که $\hat{y}_j^{(i)}$ پیش‌بینی مدل j ام برای نمونه i ام است
- وزن پیش‌بینی‌کننده با رابطه زیر محاسبه می‌شود:
 - که η نرخ آموزش است
 - مدل دقیق‌تر دارای وزن بزرگتری خواهد بود
 - اگر تقریباً تصادفی پیش‌بینی کند، وزن نزدیک به صفر خواهد بود
- سپس، وزن جدید نمونه‌ها به‌روز می‌شود
 - وزن‌ها را نرمال می‌کنیم که مجموع ۱ شود

AdaBoost

- در نهایت، پیش‌بینی‌کننده جدید با وزن‌های به‌روز شده آموزش داده می‌شود و تمام مراحل تکرار می‌شوند

- زمانی متوقف می‌شود که تعداد پیش‌بینی‌کننده تنظیم شده آموزش ببینند یا دقت مطلوب بدست بیاید

- پیش‌بینی برای نمونه‌های جدید به صورت زیر انجام می‌شود:
- که N تعداد پیش‌بینی‌کننده‌ها است

$$\hat{y}(\mathbf{x}) = \arg \max_k \sum_{j=1}^N \alpha_j (\hat{y}_j(\mathbf{x}) = k)$$

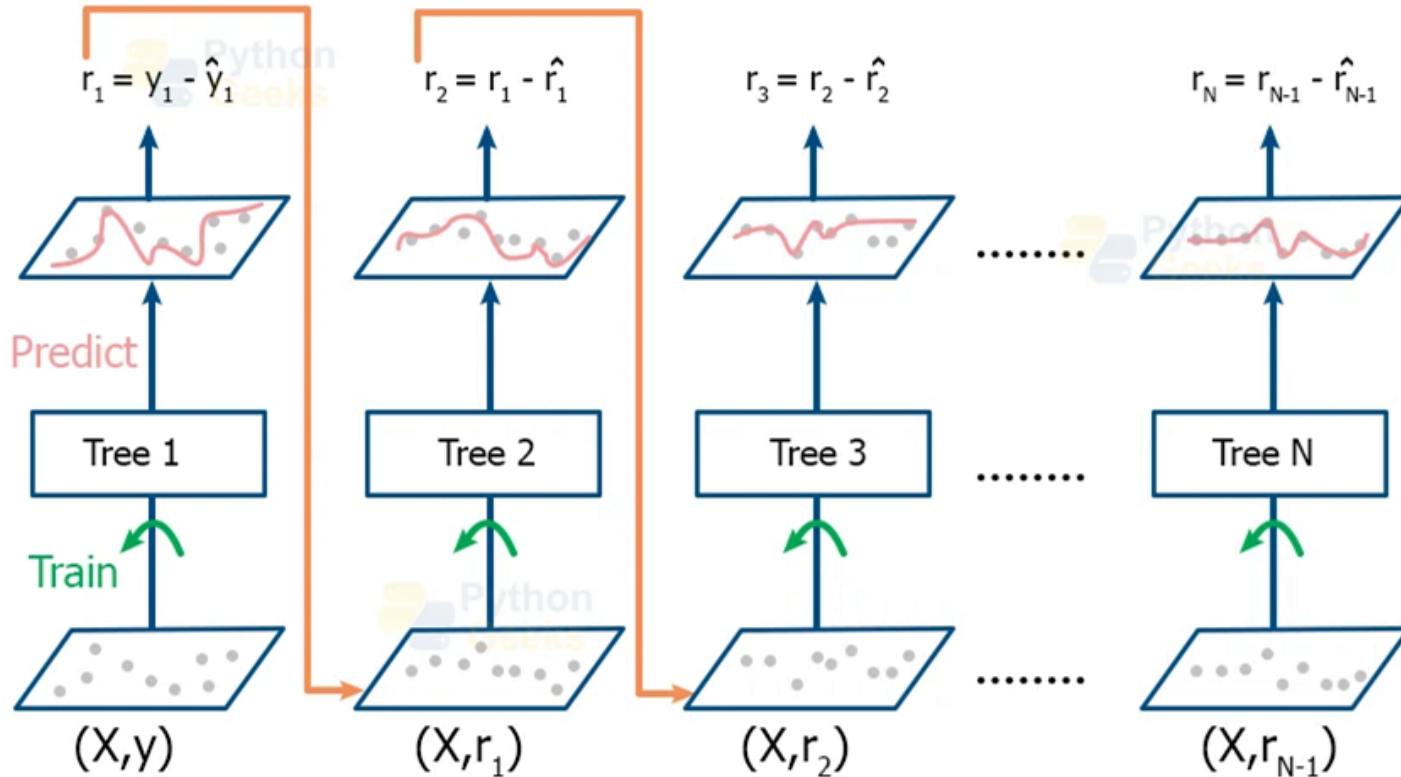
```
from sklearn.ensemble import AdaBoostClassifier
```

```
ada_clf = AdaBoostClassifier(  
    DecisionTreeClassifier(max_depth=1), n_estimators=30,  
    learning_rate=0.5, random_state=42)  
ada_clf.fit(X_train, y_train)
```

Gradient Boosting

- مشابه با AdaBoost است، اما بجای تغییر وزن نمونه‌ها در هر تکرار، پیش‌بینی‌کننده جدید را برای خطاهای باقی‌مانده از تکرارهای قبل آموزش می‌دهد

- مثال: رگرسیون



```

import numpy as np
from sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) #  $y = 3x^2 + \text{Gaussian noise}$ 

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)

y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)

y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)

>>> X_new = np.array([[ -0.4], [ 0.], [ 0.5]])
>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
array([0.49484029, 0.04021166, 0.75026781])

```

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

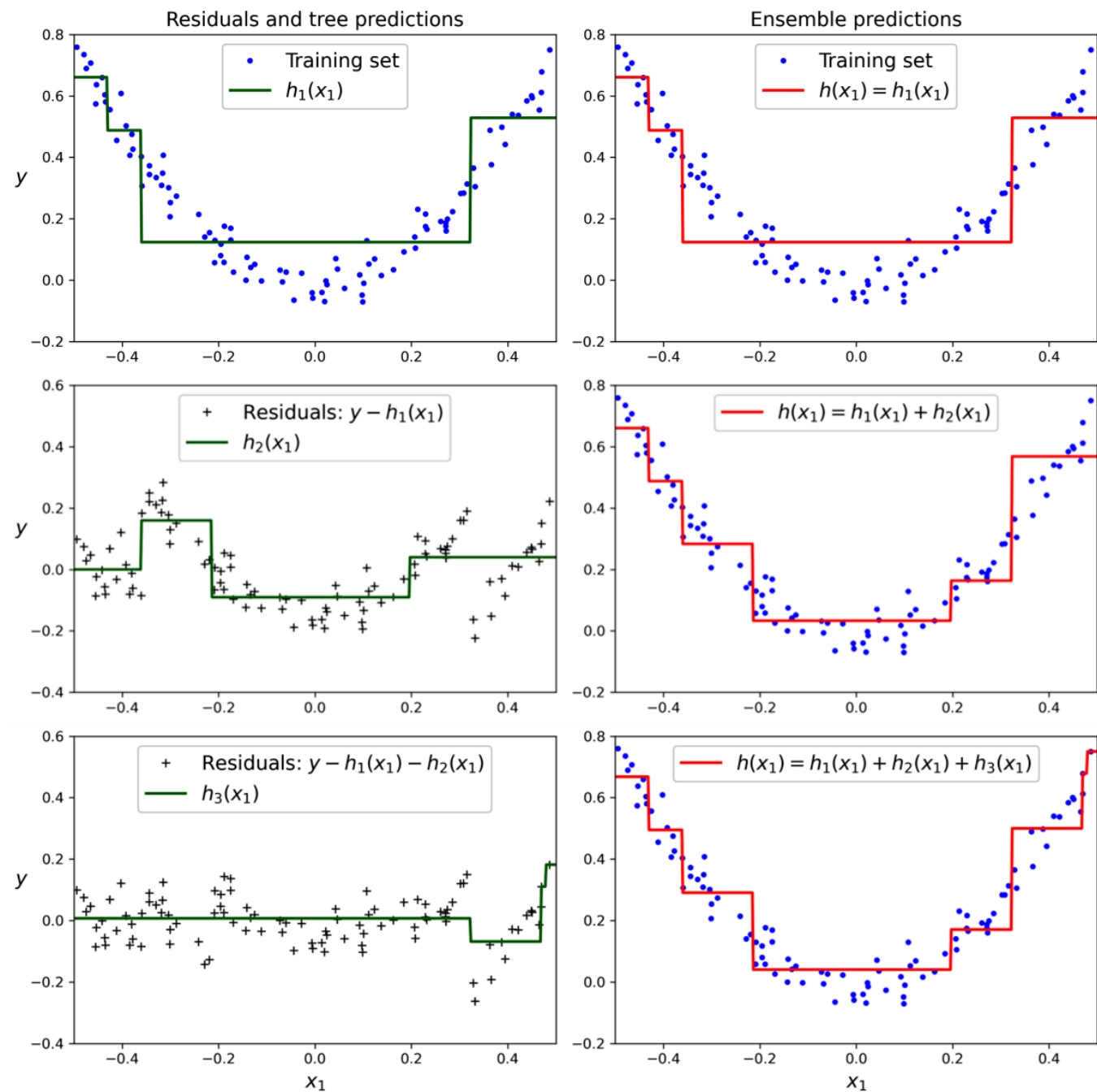
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) #  $y = 3x^2 + \text{Gaussian noise}$ 

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)

y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)

y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)

>>> X_new = np.array([[-0.4], [0.], [0.5]])
>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
array([0.49484029, 0.04021166, 0.75026781])
```



Gradient Boosting

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                  learning_rate=1.0, random_state=42)

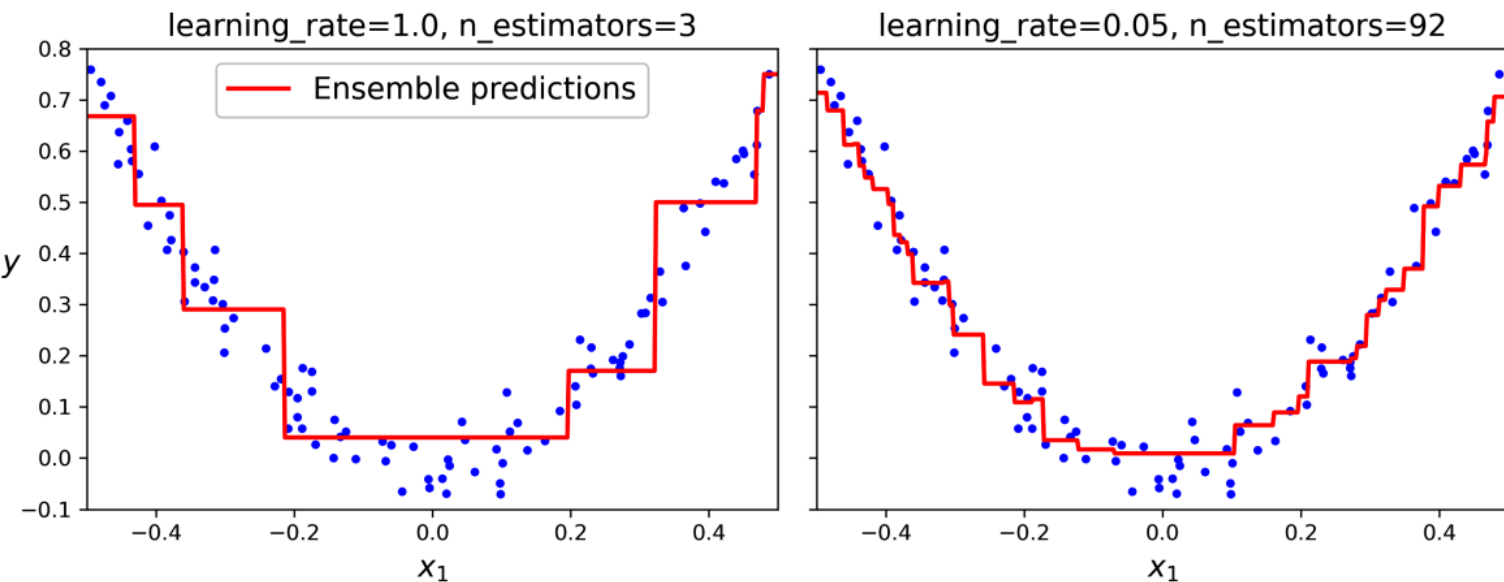
gbrt.fit(X, y)
```

- ابرپارامتر `learning_rate` تاثیر هر درخت را کنترل می کند

- اگر مقدار آن کوچک باشد، باید از درخت های بیشتری استفاده کنیم و تعمیم دهی بهتری خواهد داشت

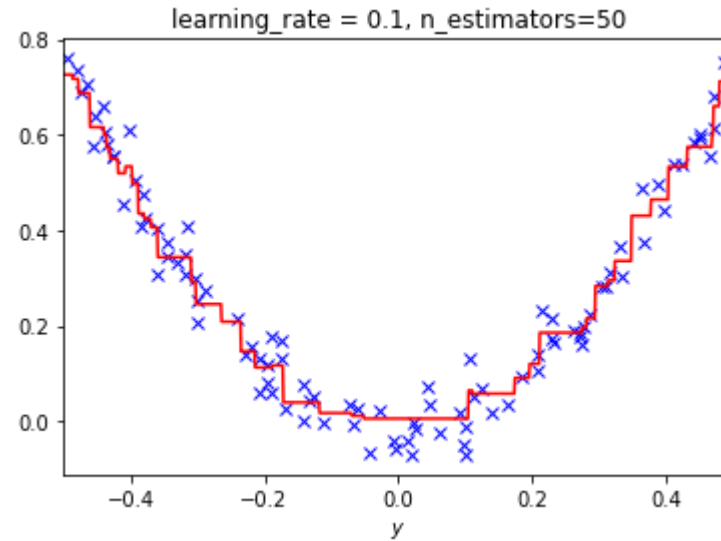
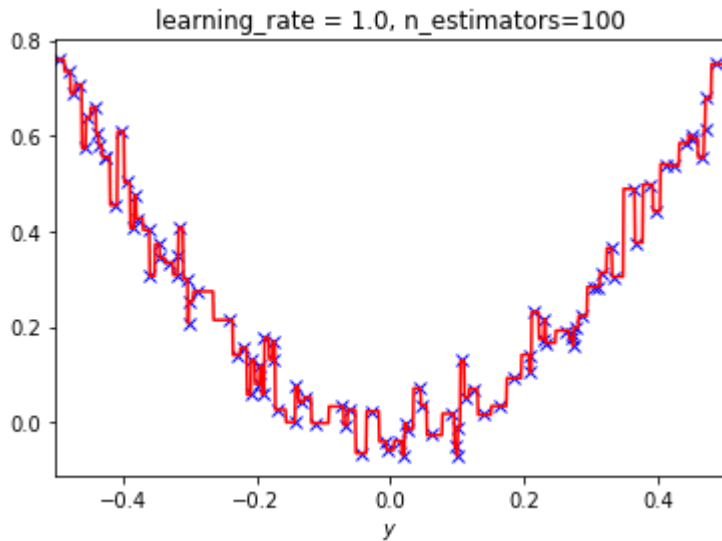
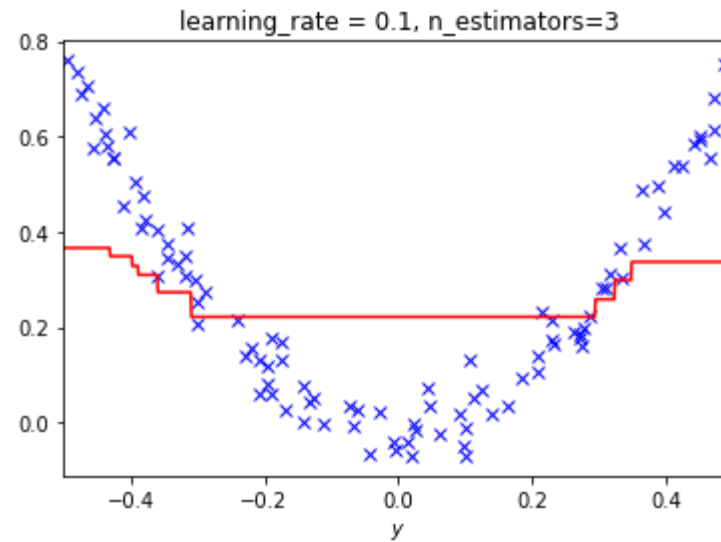
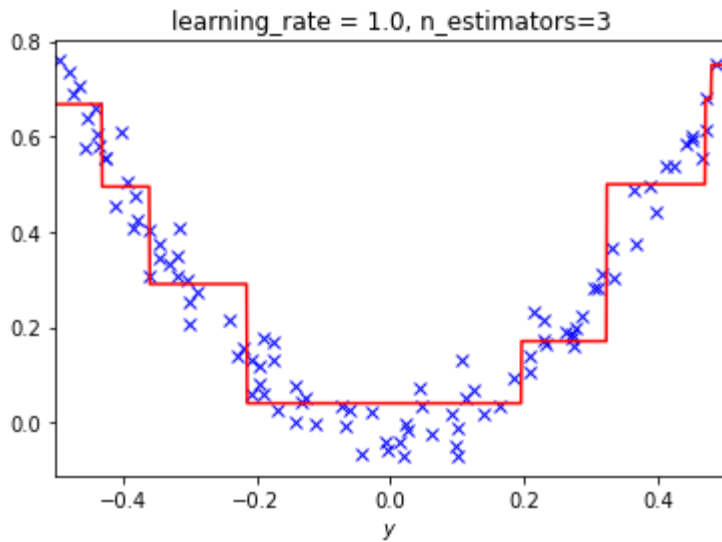
- نقش منظم سازی دارد

- این تکنیک `shrinkage` هم نامیده می شود



Gradient Boosting

- ابرپارامتر `learning_rate` تاثیر هر درخت را کنترل می کند
 - اگر مقدار آن کوچک باشد، باید از درخت های بیشتری استفاده کنیم و تعمیم دهی بهتری خواهد داشت
 - نقش منظم سازی دارد
 - این تکنیک shrinkage هم نامیده می شود



Gradient Boosting

- برای یافتن تعداد درخت بهینه، می توان از روش های جستجوی ابرپارامتر با ارزیابی متقابل استفاده کرد

- مانند GridSearchCV و RandomizedSearchCV

- با استفاده از تنظیم مقدار ابرپارامتر `n_iter_no_change` هم می توان افزودن درخت جدید را متوقف کرد
- در این حالت، به اندازه `validation_fraction` از داده ها برای اعتبارسنجی در نظر گرفته می شود و اگر به تعداد `n_iter_no_change` تکرار به اندازه `tol` بهبود حاصل نشد، ادامه کار را متوقف می کند

- می توان برای آموزش هر درخت از زیرمجموعه ای تصادفی از نمونه ها استفاده کرد که به آن stochastic gradient boosting گفته می شود

```
gbrt_best = GradientBoostingRegressor(  
    max_depth=2, learning_rate=0.05, n_estimators=500,  
    n_iter_no_change=10, random_state=42)  
gbrt_best.fit(X, y)
```

```
>>> gbrt_best.n_estimators_
```

```
92
```